

Práctico AYED1 - Sala 2 - 21-10

Práctico 4 - Ejercicio 4c

considere el siguiente programa:

2c) $y := y + y ;$
 $x := x + y$

¿Se puede escribir un programa equivalente al del ítem 2c con sólo una asignación múltiple?

Primera opción:

$y, x := y + y, x + y$

¿es equivalente al anterior? parece que no, porque el valor de x depende del valor de y :
en un caso se usa el valor de y antes de hacer " $y := y + y$ ", y en el otro caso se usa el valor de después.
pero esto **puro bla bla**, con un **contraejemplo** queda demostrada la no equivalencia:

[$y \rightarrow 6, x \rightarrow 0$]
 $y := y + y ;$
 $x := x + y$
[$y \rightarrow 12, x \rightarrow 12$]

[$y \rightarrow 6, x \rightarrow 0$]
 $y, x := y + y, x + y$

[$y \rightarrow 12, x \rightarrow 6$]

Listo, acabamos de demostrar que no son equivalentes.

Segunda opción:

$x, y := x + y, y + y$

Tampoco anda, es exactamente lo mismo que $y, x := y + y, x + y$ (repasar asignación múltiple!).

Tercera opción:

$y, x := y + y, x + (y + y)$

Idea: como en la asignación a x de " $x + y$ " queremos usar el valor nuevo de y , en realidad debemos hacer " $x + (y + y)$ ". ¿es equivalente al programa original? sí, aunque habría de demostrarlo.

¿Cuándo dos programas son equivalentes? Vimos que cuando no son equivalentes, damos un contraejemplo de un estado inicial que lleva a diferentes estados finales.

Luego, se puede entender que para que los programas sean equivalentes hay un "para todo" sobre los estados iniciales.

Podríamos decir **"para todo estado inicial la ejecución de los dos programas lleva al mismo estado final"**.

Otra versión más formal: $S1$ y $S2$ son equivalentes cuando $\{P\} S1 \{Q\}$ si y sólo si $\{P\} S2 \{Q\}$. O sea, $S1$ y $S2$ "cumplen y dejan de cumplir las mismas ternas de Hoare."

Práctico 6 - Ejercicio 1a

Calcular wp.

```
Var x, y : Num;  
{ P }  
x := x + y  
{ x = 6  $\wedge$  y = 5 }
```

¿qué cosas podría ser P para que la terna valga?

- $x = 1 \wedge y = 5$
- False (recordemos: la terna "{ False } S { Q }" siempre vale.)

De estas dos, ¿cuál es la más débil? o sea, ¿cuál es la que más posibilidades incluye?
" $x = 1 \wedge y = 5$ ".

Ahora usando la wp:

```
wp.S.Q  
= { sustituyo S y Q }  
wp.(x := x + y).(x = 6  $\wedge$  y = 5)  
= { definición de wp para la asignación }  
(x = 6  $\wedge$  y = 5)(x  $\leftarrow$  x + y) // esto se lee: " $x = 6 \wedge y = 5$ " pero reemplazando x por " $x + y$ ".  
= { hacemos el reemplazo }  
x + y = 6  $\wedge$  y = 5  
= { leibniz 2 (reemplazo de iguales por iguales) }  
x + 5 = 6  $\wedge$  y = 5
```

= { arit. }
 $x = 1 \wedge y = 5$

Listo.

Ejercicio Inventado

```
Var x : Int ;  
{ P }  
  x := x * x  
{ x ≥ 9 }
```

¿Qué cosas puede ser P para que valga la terna?:

- False
- $x = 100$
- $x = -123$
- $x = 100 \vee x = -123$
- $x \geq 25 \wedge \text{espar}.x$
- $x \geq 3$
- $x \leq -10000008$
- $x \geq 3 \vee x \leq -3$, lo mismo que $|x| \geq 3$.

¿Cuál es el más débil de todos? El último: $|x| \geq 3$.

¿Podremos llegar a eso con la wp?

$\text{wp}.(x := x * x).(x \geq 9)$

$= \{ \text{def. wp para } := \}$
 $(x \geq 9)(x \leftarrow x * x)$
 $= \{ \text{sust.} \}$
 $x * x \geq 9$
 $= \{ 9 = 3 * 3 \}$
 $x * x \geq 3 * 3$
 $= \{ \text{teorema: "a}^2 \geq b^2 \text{ si y sólo si } |a| \geq |b| \} \}$
 $|x| \geq |3|$
 $= \{ \text{algebra} \}$
 $|x| \geq 3$

(habrá otras formas de probarlo también)

Práctico 6 - Ejercicio 1e

Var $x, y, a : \text{Num} ;$
 $\{ P \}$
 $a, x := x, y ;$
 $\{ Q \}$
 $y := a$
 $\{ R : x = B \wedge y = A \}$

La consigna es dar P y Q usando wp. ¿Cómo los calculamos?

(1) Q debe ser $\text{wp.}(y := a).R$

Si vemos que la terna para P es : $\{ P \} a, x := x, y ; y := a \{ R \}$ vemos que:

(2) P debe ser $\text{wp.}(a, x := x, y ; y := a).R$

Aunque también vemos que P tiene la terna: $\{ P \} a, x := x, y \{ Q \}$, o sea que:

(3) P debe ser $\text{wp.}(a, x := x, y).Q$
es decir $\text{wp.}(a, x := x, y).(\text{wp.}(y := a).R)$ (esto por (1))

Luego (2) y (3) deberían ser equivalentes:

$\text{wp.}(a, x := x, y ; y := a).R$ es lo mismo que
 $\text{wp.}(a, x := x, y).(\text{wp.}(y := a).R)$

Está bien esto ya que de hecho esta es la definición de la wp para la secuenciación “;”.

Vamos a las cuentas: Primero Q:

$\text{wp.}(y := a).R$
 $= \{ \text{def wp para } := \}$
 $(x = B \wedge y = A)(y \leftarrow a)$
 $= \{ \text{sust.} \}$
 $x = B \wedge a = A$

Listo Q. Ahora P: ¿Usamos (2) o (3)? Haremos (3) ya que tenemos Q y nos ahorramos trabajo.

$\text{wp.}(a, x := x, y).Q$

$= \{ \text{def. wp para } := \}$
 $(x = B \wedge a = A)(a \leftarrow x, x \leftarrow y)$
 $= \{ \text{sust.} \}$
 $y = B \wedge x = A$

Listo P! Sólo para completar, veamos si usábamos (2) qué pasaba:

$\text{wp.}(a, x := x, y ; y := a).R$
 $= \{ \text{def. wp para " ; " } \}$
 $\text{wp.}(a, x := x, y).(\text{wp.}(y := a).R)$
 $= \{ \text{ya vimos wp.}(y := a).R \}$
 $\text{wp.}(a, x := x, y).(x = B \wedge a = A)$
 $= \{ \text{ya lo hicimos también} \}$
 $y = B \wedge x = A$

Así queda el programa con las anotaciones:

Var $x, y, a : \text{Num}$;
 $\{ P : y = B \wedge x = A \}$
 $a, x := x, y ;$
 $\{ Q : x = B \wedge a = A \}$
 $y := a$
 $\{ R : x = B \wedge y = A \}$

¿Qué hace este programa? Miremos cómo se relaciona la pre y la post.

Al principio x valía A e y valía B , y al final es al revés, x vale B e y vale A , así que este programa
 “Intercambia los valores de x e y ”

Nunca confundir el **qué hace** con el **cómo**. El “cómo” es el programa en sí, pero hay muchas formas de hacer este intercambio. Por ejemplo otro programa que hace lo mismo pero de otra forma es:

$x, y := y, x$

Práctico 6 - Ejercicio 1f

```
Var x, y : Num ;  
{ P }  
if  $x \geq y \rightarrow$   
  { Q1 }  
   $x := 0$   
  { R1 }  
 $\square x \leq y \rightarrow$   
  { Q2 }  
   $x := 2$   
  { R2 }  
fi  
{ T :  $(x = 0 \vee x = 2) \wedge y = 1$  }
```

¿Qué llenamos primero acá?

Q1 será $\text{wp.}(x := 0).R1$ o sea que para saber Q1 necesito R1.

Q2 será $\text{wp.}(x := 2).R2$ o sea que para saber Q2 necesito R2.

¿Qué son R1 y R2? Son exactamente T.

$$R1 \equiv (x = 0 \vee x = 2) \wedge y = 1$$

$$R2 \equiv (x = 0 \vee x = 2) \wedge y = 1$$

Ya podemos ver Q1:

$$\text{wp.}(x := 0).R1$$

$$= \{ \text{def. wp para } := \}$$

$$((x = 0 \vee x = 2) \wedge y = 1)(x \leftarrow 0)$$

$$= \{ \text{sust.} \}$$

$$(0 = 0 \vee 0 = 2) \wedge y = 1$$

$$= \{ \text{lógica} \}$$

$$(\text{True} \vee 0 = 2) \wedge y = 1$$

$$= \{ \text{más lógica} \}$$

$$y = 1$$

Y Q2:

$$\text{wp.}(x := 2).R1$$

$$= \{ \text{def. wp para } := \}$$

$$((x = 0 \vee x = 2) \wedge y = 1)(x \leftarrow 2)$$

$$= \{ \text{sust.} \}$$

$$(2 = 0 \vee 2 = 2) \wedge y = 1$$

$$= \{ \text{lógica} \}$$

$$(2 = 0 \vee \text{True}) \wedge y = 1$$

$$= \{ \text{más lógica} \}$$

$$y = 1$$

Sólo falta P. ¿Qué es P? Es la wp de un **if**:

P es $\text{wp.}(\text{if } x \geq y \rightarrow x := 0 \ \square \ x \leq y \rightarrow x := 2 \text{ fi}).T$

Calculemos:

$\text{wp.}(\text{if } x \geq y \rightarrow x := 0 \ \square \ x \leq y \rightarrow x := 2 \text{ fi}).T$
= { def. wp para el "if" }
 $(x \geq y \ \vee \ x \leq y)$
 $\wedge (x \geq y \Rightarrow \text{wp.}(x := 0).T)$
 $\wedge (x \leq y \Rightarrow \text{wp.}(x := 2).T)$
= { R1 y R2 son T }
 $(x \geq y \ \vee \ x \leq y)$
 $\wedge (x \geq y \Rightarrow \underline{\text{wp.}(x := 0).R1})$
 $\wedge (x \leq y \Rightarrow \underline{\text{wp.}(x := 2).R2})$
= { esas wp son Q1 y Q2 que ya las calculamos recién }
 $(x \geq y \ \vee \ x \leq y)$
 $\wedge (x \geq y \Rightarrow \underline{Q1})$
 $\wedge (x \leq y \Rightarrow \underline{Q2})$
= { reemplazamos Q1 y Q2 }
 $(x \geq y \ \vee \ x \leq y)$
 $\wedge (x \geq y \Rightarrow y = 1)$
 $\wedge (x \leq y \Rightarrow y = 1)$
= { logica }
True
 $\wedge (x \geq y \Rightarrow y = 1)$
 $\wedge (x \leq y \Rightarrow y = 1)$
= { logica }
 $(x \geq y \Rightarrow y = 1) \wedge (x \leq y \Rightarrow y = 1)$
= { sin importar lo q pase, $y = 1$ } versión poco formal

y = 1

= { $(P \Rightarrow R) \wedge (Q \Rightarrow R)$ es lo mismo que $P \vee Q \Rightarrow R$ (seguro está demostrado por ahí) }

$x \geq y \vee x \leq y \Rightarrow y = 1$

= { lógica }

True $\Rightarrow y = 1$

= { lógica }

y = 1

Esto es P. Al final queda el programa con las anotaciones:

Var x, y : Num ;

{ P : y = 1 }

if $x \geq y \rightarrow$

{ Q1 : y = 1 }

x := 0

{ R1 : $(x = 0 \vee x = 2) \wedge y = 1$ }

$\square$ $x \leq y \rightarrow$

{ Q2 : y = 1 }

x := 2

{ R2 : $(x = 0 \vee x = 2) \wedge y = 1$ }

fi

{ T : $(x = 0 \vee x = 2) \wedge y = 1$ }